

Task Píxel (5000)



Versión: 0.5

Fecha: 04/08/2026

Índice

Índice.....	2
Objetivo del Proyecto.....	4
Usuario Objetivo.....	4
Requisitos Funcionales	4
Requisitos No Funcionales	5
Arquitectura Inicial.....	5
Capas	5
Estructura de Carpetas.....	7
Guía de Identidad Visual.....	8
Estilo General	8
Paleta de colores	8
Tipografía	8
Iconografía.....	9
Componentes.....	9
Animaciones.....	9
Recursos Gráficos	10
Filosofía de Diseño	10
Plan de Desarrollo dividido por versiones.....	10
v0.1 - Inicio del proyecto.....	10
v0.2 - Base funcional.....	10
v0.3 - Inicio del proyecto.....	11
v0.4 - Persistencia.....	11
v0.5 - Interfaz	11
v0.6 - Pruebas.....	12
v0.7 - Documentación.....	12
v1.0 - Publicación	12
Roadmap.....	13
Configuración	13
Desarrollo.....	13
Documentación.....	13
Publicación.....	14

Por: Luis Alberto Hernández López

Historial de Documento.....	14
Tecnologías	14
Tecnología	14
Riesgos.....	15
Riesgos identificados.....	15

Objetivo del Proyecto

Desarrollar una aplicación de escritorio que permita a los usuarios administrar sus tareas personales mediante la creación, edición, priorización, marcado como completadas y eliminación de tareas, utilizando una interfaz inspirada en Pixel Art.

Usuario Objetivo

Cualquier persona que necesite o desee tomar registro de sus tareas, ayudando a la persona a registrar, organizar e incluso tachar tareas ya realizadas. Para las personas que necesiten organización en su vida diaria, como podrían ser;

- Estudiantes.
- Profesionistas.
- Personas que desean organizar actividades.

Nivel técnico:

- Básico

Sistema operativo:

- Windows

Edad estimada:

- 15 años en adelante

Requisitos Funcionales

1. El usuario podrá escribir la descripción de su tarea.
2. El usuario tendrá 2 botones a disposición para que el usuario pueda crear nuevas tareas y eliminarlas.
3. El usuario podrá elegir el orden de prioridad de la tarea teniendo en cuenta si es;
 - a. Urgente
 - b. Importante
 - c. Cotidiano
4. Para las tareas ya realizadas se tendrá la opción de tacharla.

5. La interfaz del sistema puede alternar entre 3 colores a preferencia del usuario.

Requisitos No Funcionales

1. El programa tarda menos de 3 segundos en iniciar.
2. Funciona sin necesidad de conexión a internet.
3. El programa funciona de manera fluida y sin atascos en cuanto es abierta.
4. Se presenta un programa sencillo con código limpio y documentado.

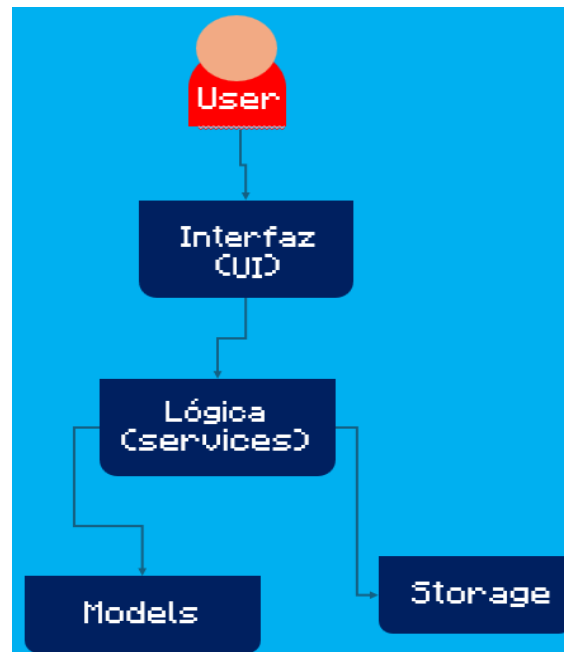
Arquitectura Inicial

Capas:

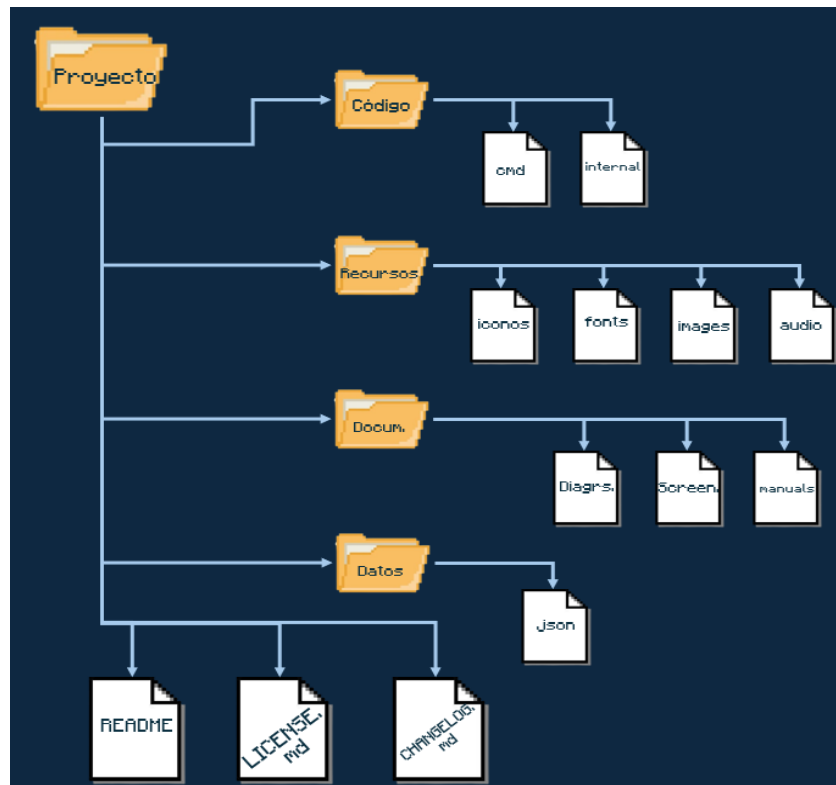
1. **Interfaz de Usuario (UI):** Responsable de la interacción con el usuario.
 - Muestra la información al usuario
 - Recibe las acciones del usuario
2. **Servicios:** Contiene las reglas de funcionamiento del programa.
 - Procesar la información
 - Validar datos
 - Coordinar las operaciones del sistema
3. **Modelos:** Representan las entidades principales del proyecto.
 - Definir la estructura de los datos
 - Facilitar el intercambio de información entre las demás capas
4. **Storage:** Responsable del almacenamiento y recuperación de la información.
 - Leer datos
 - Guardar datos
 - Administrar archivos o bases de datos
5. **Recursos:** Almacena todos los recursos utilizados por la aplicación.
 - Iconos

Por: Luis Alberto Hernández López

- Imágenes
- Tipografías
- Sonidos
- Animaciones



Estructura de Carpetas



Carpeta o Archivo	Propósito
cmd	Punto de entrada de la aplicación.
internal	Código principal del proyecto.
ui	Componentes de la interfaz de usuario.
services	Lógica de negocio.
models	Definición de las estructuras de datos.
storage	Lectura y escritura de archivos o bases de datos.
assets	Recursos gráficos, tipografías y sonidos.
data	Archivos de datos utilizados por la aplicación.
docs	Documentación técnica y manuales
test	Pruebas del proyecto.

Guía de Identidad Visual

Estilo General

El proyecto utilizará una interfaz inspirada en el píxel art. Todos los elementos gráficos seguirán un estilo Píxel Art con una apariencia sencilla, amigable y consistente. Se evitarán elementos fotorrealistas o diseños minimalistas

Paleta de colores

Para este proyecto se utilizará la Paleta de colores Pear36.

Elemento		Color		
Fondo principal		#ffffeb		
Paneles (cuadro de terminado)		#473b78		
Texto		#473b78		
Confirmación		#4da6ff		
Advertencia		#f2a65e		
Error		#ff6b97		
#5e315b	#8c3f5d	#ba6156	#f2a65e	#ffe478
#cfff70	#8fde5d	#3ca370	#3d6e70	#323e4f
#322947	#473b78	#4b5bab	#4da6ff	#66ffe3
#ffffeb	#c2c2d1	#7e7e8f	#606070	#43434f
#272736	#3e2347	#57294b	#964253	#e36956
#ffb570	#ff9166	#eb564b	#b0305c	#73275c
#422445	#5a265e	#80366b	#bd4882	#ff6b97
#ffb5b5				

Tipografía

- **Títulos:** Minecraftia
- **Subtítulos:** Minecraftia
- **Texto:** Arial
- **Código:** Cascadia Code

Iconografía

- Todos los iconos serán elaborados en Aseprite.
- Mantendrán el mismo estilo Píxel Art.
- Tamaño base 16x16 px.
- Podrán escalarse a 32x32 px sin perder consistencia.

Componentes

Botones

- Bordes pixelados.
- Sombra inferior de un píxel.
- Cambian ligeramente de color al pasar el cursor
- Se hunden un píxel al hacer clic.

Ventanas

- Bordes gruesos.
- Esquinas cuadradas.
- Barra superior personalizada.

Paneles

- Fondo ligeramente más claro que el fondo principal.
- Borde oscuro de un píxel.

Animaciones

Las animaciones deberán ser cortas (150–300 ms), discretas y utilizadas únicamente para reforzar la interacción del usuario. Se evitarán animaciones excesivas que distraigan del contenido principal.

Recursos Gráficos

Definir de donde salen los recursos.

Recursos	Origen
Iconos	Aseprite
Ilustraciones	Aseprite
Tipografía Pixel	Minecraftia
Sonidos	Recursos libres

Filosofía de Diseño

El objetivo del diseño no es impresionar mediante efectos visuales complejos, sino ofrecer una experiencia agradable, clara y consistente. La interfaz debe transmitir una sensación de estar utilizando una aplicación artesanal, cuidada, cómoda y con personalidad propia. Cada elemento gráfico debe tener una función clara y contribuir a una experiencia sencilla e intuitiva

Plan de Desarrollo dividido por versiones

v0.1 – Inicio del proyecto

Objetivo: Preparar el entorno de desarrollo.

Actividades:

- Crear repositorio.
- Configurar Git.
- Crear la estructura de carpetas.
- Crear README inicial.
- Crear el documento de diseño.
- Configurar el proyecto en Go.

Resultado esperado: El proyecto puede compilar y cuenta con la estructura base.

v0.2 – Base funcional

Por: Luis Alberto Hernández López

Objetivo: Crear las estructuras principales del proyecto.

Actividades:

- Crear modelos.
- Crear lógica inicial.
- Crear interfaz básica.
- Crear almacenamiento

Resultado esperado: El programa ya tiene una estructura funcional, aunque aún no implemente todas las características.

v0.3 – Inicio del proyecto

Objetivo: Implementar funciones esenciales.

Actividades:

- Agregar registros.
- Eliminar registros.
- Modificar registros.
- Buscar registro.

Resultado esperado: El usuario puede utilizar la funcionalidad principal del programa.

v0.4 – Persistencia

Objetivo: Guardar la información.

Actividades:

- Leer archivos.
- Guardar archivos.
- Validaciones.

Resultado esperado: La información permanece entre ejecuciones.

v0.5 – Interfaz

Objetivo: Mejorar la experiencia del usuario

Por: Luis Alberto Hernández López

Actividades:

- Mejorar diseño.
- Agregar iconos.
- Ajustar botones.
- Mejorar navegación.

Resultado esperado: La aplicación resulta cómoda de utilizar.

v0.6 – Pruebas

Objetivo: Detectar errores.

Actividades:

- Corregir bugs.
- Probar funcionalidades.
- Revisar validaciones.

Resultado esperado: El proyecto funciona correctamente

v0.7 – Documentación

Objetivo: Completar la documentación.

Actividades:

- README.
- Capturas
- Manual
- Diagramas

Resultado esperado: Cualquier persona puede entender el proyecto.

v1.0 – Publicación

Objetivo: Liberar la primera versión estable.

Actividades:

- Crear Release.

Por: Luis Alberto Hernández López

- Etiquetar versión.
- Publicar en GitHub.
- Última revisión.

Resultado esperado: Proyecto finalizado.

Roadmap

Configuración

- ☐ Crear repositorio
- ☐ Configurar Git
- ☐ Crear README
- ☐ Crear estructura de carpetas
- ☐ Configurar Go

Desarrollo

- ☐ Crear modelos
- ☐ Crear lógica
- ☐ Crear interfaz
- ☐ Agregar persistencia
- ☐ Validar datos

Documentación

- ☐ Completar README
- ☐ Crear manual de usuario
- ☐ Crear manual de desarrollador
- ☐ Agregar capturas
- ☐ Crear diagramas

Publicación

- ☐ Crear Release
- ☐ Publicar en GitHub
- ☐ Etiquetar versión
- ☐ Actualizar CHANGELOG

Historial de Documento

Versión	Fecha	Cambios
0.1	13/07/2026	Documento inicial
0.2	24/07/2026	Se añadieron temas principales
0.3	30/07/2026	Se termino con la idea principal del documento
0.4	31/07/2026	Se corrigieron errores de documentación y se añadió un índice
0.5	1/08/2026	Se añadieron los temas Historial de documento, Tecnologías y Riesgos
0.6	4/08/2026	Se añadió logo e icono del proyecto y se actualizo la portada

Tecnologías

Tecnología	Uso
Go	Desarrollo
Git	Control de versiones
GitHub	Repositorio
Aseprite	Recursos gráficos

Riesgos

Riesgos identificados

- Aprendizaje de interfaz gráfica.
- Diseño de recursos Píxel Art.
- Falta de experiencia con persistencia.